Images & Multiple Data Types

Theory Session: Pillow, Image Properties

Mr. Jayakumar T V

Department of Avionics
Indian Institute of Space Science and Technology

Duration: 1 h 15 min

Opening story: two moves that changed how we see AI

AlphaGo vs. Lee Sedol, Seoul, March 2016 (AlphaGo won 4–1)

Game 2, Move 37 (AlphaGo)

- **Play:** early *shoulder hit* on the fifth line, in the open centre.
- **Reaction:** experts were stunned and first suspected a mistake; humans are taught not to play so high so early.
- **Impact:** AlphaGo estimated a human would play it with probability $\approx 1/10{,}000$. It shaped the whole game and is cited as machine "creativity".

Game 4, Move 78 (Lee Sedol)

- **Play:** a brilliant *wedge* in the middle, called "God's touch".
- **Glitch:** AlphaGo also rated it $\approx 1/10{,}000$; its evaluation went wrong and it played weak moves.
- **Impact:** Lee Sedol's only win, showing human creativity can still surprise the machine.

Each side produced a move the other had judged a 1-in-10,000 event.

Why this matters: growing human–AI interaction

- **From rivals to collaborators:** professionals now study AI moves and have changed opening ideas; the machine became a *teacher*.
- **Humans still add value:** Move 78 shows that human insight can expose a machine's blind spot.
- **Data drives it:** AlphaGo learned from human games, then from self-play.
- **Trust and limits:** we must learn when to rely on AI and when to question it.

Link to today's topic

A Go board is a 19×19 grid (361 points). AlphaGo's networks read it like an **image**: stacked 19×19 input planes.

Today's AI reads images, text and tables together. This session builds the skills to prepare such data.

Session Plan (75 minutes)

No	Block	You will
1	Opening story & discussion	hear the AlphaGo story, discuss
2	Why images & mixed data	see real examples
3	Images are numbers	predict array shapes
4	Pillow function guide	learn names, purposes, return values
5	Image properties	formats, metadata, quick quiz
6	Wrap-up & lab preview	questions

Real data is rarely a single type

- **Medical**: scan + patient record + report text
- **E-commerce**: product photo + title + price + reviews
- **Remote sensing**: satellite tile + coordinates + date + land-cover label
- **Social media**: image + caption + hashtags
- **Documents**: scanned page + OCR text + table

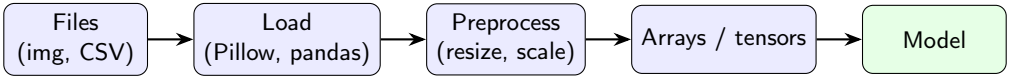
Key idea

A model that sees only one modality ignores information that the other modalities already carry.

Practical consequence

Before modelling, we must *load, align and clean* images and tabular/text data together.

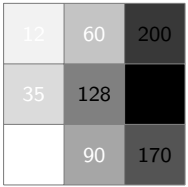
Where Python fits in



- **Pillow** handles the image side (decode, inspect, transform, encode).
- **pandas** handles the table/text side.
- **NumPy** is the bridge that turns both into numbers.

An image is a grid of numbers

- Every small square is a **pixel**.
- A pixel stores a number: 0 = black, 255 = white.
- A grayscale image = rows $\times$ columns of numbers.
- A colour image has **channels**: shape $H \times W \times C$.



A 3×3 grayscale image

Colour pixels and Pillow “modes”

A colour pixel is **(R, G, B)**, each from 0 to 255.

Colour	(R, G, B)
Red	(255, 0, 0)
Green	(0, 255, 0)
Blue	(0, 0, 255)
Yellow	(255, 255, 0)
White	(255, 255, 255)
Black	(0, 0, 0)

Mode	Bands	Meaning
`"1"`	1	black/white
`"L"`	1	grayscale
`"RGB"`	3	true colour
`"RGBA"`	4	colour + transparency
`"CMYK"`	4	print
`"P"`	1	palette (GIF)

Rule

Convert to one known mode (usually RGB) before making arrays, so all shapes match.

Try it 1: images as numbers

1. An RGB image is 640 wide and 480 high. Array shape?

Try it 1: images as numbers

1. An RGB image is 640 wide and 480 high. Array shape? $(480, 640, 3)$
2. A grayscale (`"L"`) image is 100 wide, 50 high. Array shape?

Try it 1: images as numbers

1. An RGB image is 640 wide and 480 high. Array shape? $(480, 640, 3)$
2. A grayscale (`"L"`) image is 100 wide, 50 high. Array shape? $(50, 100)$, no colour axis
3. How many bytes does the 640×480 RGB image need (1 byte per value)?

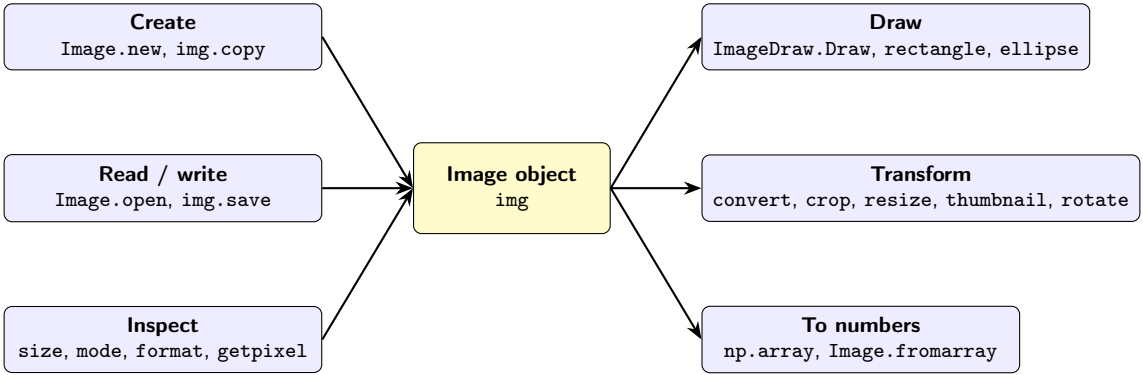
Try it 1: images as numbers

1. An RGB image is 640 wide and 480 high. Array shape? $(480, 640, 3)$
2. A grayscale (`"L"`) image is 100 wide, 50 high. Array shape? $(50, 100)$, no colour axis
3. How many bytes does the 640×480 RGB image need (1 byte per value)?
   $640 \cdot 480 \cdot 3 = 921{,}600$ bytes ≈ 0.88 MiB
4. Which colour is $(255, 255, 0)$?

Try it 1: images as numbers

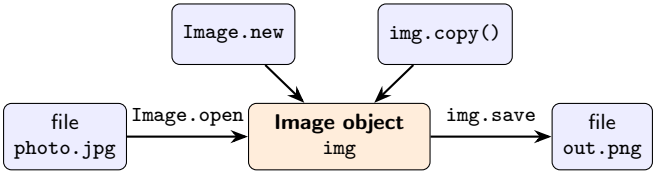
1. An RGB image is 640 wide and 480 high. Array shape? $(480, 640, 3)$
2. A grayscale (`"L"`) image is 100 wide, 50 high. Array shape? $(50, 100)$, no colour axis
3. How many bytes does the 640×480 RGB image need (1 byte per value)?
   $640 \cdot 480 \cdot 3 = 921{,}600$ bytes ≈ 0.88 MiB
4. Which colour is $(255, 255, 0)$? Yellow (red + green light)

Pillow at a glance: six families of functions



Almost everything in Pillow is either a function that **makes** an image, **asks** about it, **changes** it, or **exports** it.

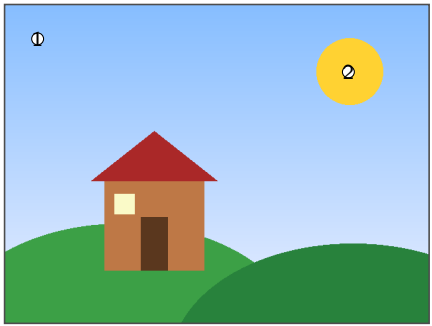
Read and write: Image.open, save, new, copy



Function	Purpose	Returns
`Image.open(path)`	read an image file; pixels load when first needed	an `Image`
`img.save(path)`	write to disk; the ending (`.png`, `.jpg`) chooses the format	nothing
`Image.new(mode, size, colour)`	blank image of a given mode and size	an `Image`
`img.copy()`	duplicate, to protect the original from in-place changes	an `Image`

Inspect: size, mode, format, getpixel

Name	Purpose / returns
`img.size`	**(width, height)** in pixels, e.g. `(640, 480)`
`img.width`, `img.height`	the two numbers separately
`img.mode`	pixel type: `"RGB"`, `"L"`, ...
`img.format`	file type when opened from a file (`"PNG"`); `None` for new images
`img.getpixel((x, y))`	value at one pixel: a tuple for RGB, a number for `"L"`

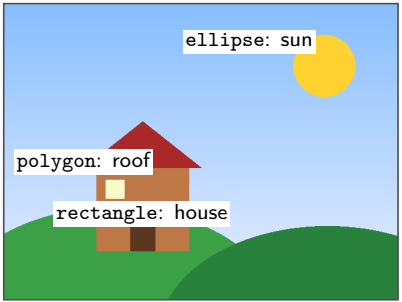


`getpixel((50, 50))` → (145, 195, 255) sky
`getpixel((520, 100))` → (255, 210, 50) sun
x goes across, y goes down from the top-left.

Draw: ImageDraw.Draw, rectangle, ellipse

Function	Purpose
`ImageDraw.Draw(img)`	creates a "pen" tied to `img`; drawing changes that image
`d.rectangle(box, fill=c)`	filled rectangle inside the box
`d.ellipse(box, fill=c)`	filled ellipse inside the box (a circle if the box is square)
`d.line`, `d.polygon`, `d.text`	also available: lines, shapes with many corners, text

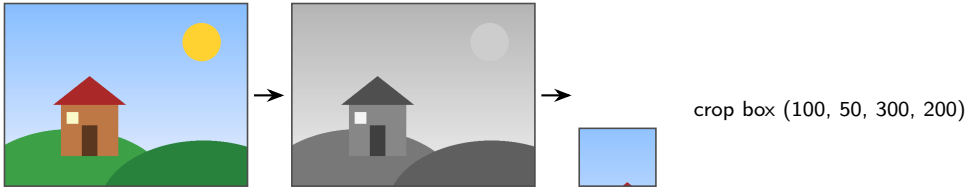
box = (left, top, right, bottom), in pixels.



The demo picture was drawn with only these functions.

Transform A: convert and crop

Function	Purpose and why	Returns
`img.convert(mode)`	change the pixel type, e.g. to `"L"` (gray: 1 number per pixel) or `"RGB"` (make all images alike)	a new `Image`
`img.crop((left, upper, right, lower))`	cut out a rectangle, e.g. the region of interest; width = right − left, height = lower − upper	a new `Image`



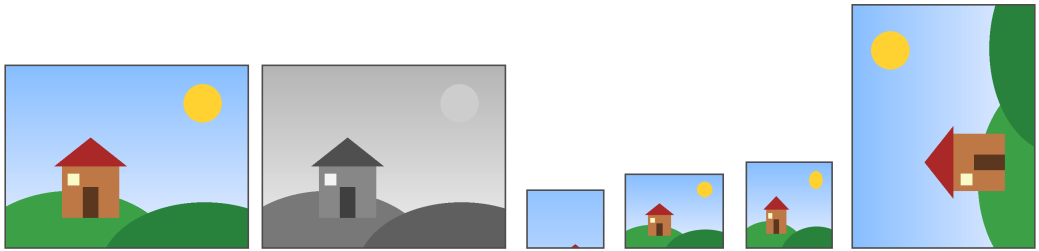
Transform B: `resize`, `thumbnail`, `rotate`

Function	Purpose and why	Returns
`img.resize((w, h))`	exact new size, so inputs have the same size; may stretch	a new `Image`
`img.thumbnail((w, h))`	shrink to fit inside the box, keeping proportions; **changes img in place**	nothing
`img.rotate(angle, expand=True)`	turn counter-clockwise; `expand=True` enlarges the canvas so nothing is cut off	a new `Image`
`img.transpose(...)`	mirror (left-right / top-bottom)	a new `Image`

Careful

`thumbnail` returns nothing and edits `img` itself. Call `img.copy()` first if you still need the original.

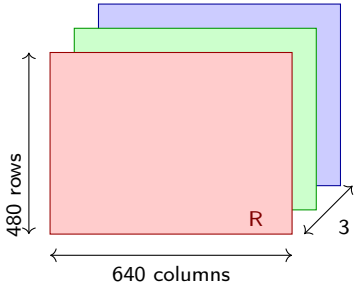
All transforms of one picture, drawn to the same scale



Scale: 200 pixels = 1 cm. `thumbnail((256, 256))`: factor $256/640 = 0.4$, so $480 \cdot 0.4 = 192$. Without `expand=True`, `rotate(90)` keeps 640×480 and cuts the corners (checked in Pillow).

Bridge to numbers: `np.array` and `Image.fromarray`

Function	Purpose / returns
`np.array(img)`	image → NumPy array of shape (H, W, C), type `uint8`
`arr.shape`	the shape: **height first**
`arr.astype("float32")`	change the number type, e.g. before dividing by 255
`Image.fromarray(arr)`	array → image (expects `uint8`)



`img.size`=(640, 480) but `shape`=(480, 640, 3).

Why scale to $[0, 1]$?

Models train better on small numbers. Example (ImageNet red channel: mean 0.485, std 0.229): value $200 \to 200/255 = 0.7843 \to (0.7843 - 0.485)/0.229 \approx 1.307$.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture. `Image.new("RGB", (640, 480), (255, 255, 255))`
2. Read the file `holiday.jpg`.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture. `Image.new("RGB", (640, 480), (255, 255, 255))`
2. Read the file `holiday.jpg`. `Image.open("holiday.jpg")`
3. Find out whether an image is gray or colour.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture. `Image.new("RGB", (640, 480), (255, 255, 255))`
2. Read the file `holiday.jpg`. `Image.open("holiday.jpg")`
3. Find out whether an image is gray or colour. `img.mode` (`"L"` or `"RGB"`)
4. Make a small preview that keeps the shape.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture. `Image.new("RGB", (640, 480), (255, 255, 255))`
2. Read the file `holiday.jpg`. `Image.open("holiday.jpg")`
3. Find out whether an image is gray or colour. `img.mode` (`"L"` or `"RGB"`)
4. Make a small preview that keeps the shape. `img.thumbnail(...)`, after `img.copy()`
5. Turn the picture into numbers for a model.

Try it 2: which function would you call?

1. Make a blank white 640×480 picture. `Image.new("RGB", (640, 480), (255, 255, 255))`
2. Read the file `holiday.jpg`. `Image.open("holiday.jpg")`
3. Find out whether an image is gray or colour. `img.mode` (`"L"` or `"RGB"`)
4. Make a small preview that keeps the shape. `img.thumbnail(...)`, after `img.copy()`
5. Turn the picture into numbers for a model. `np.array(img)`

Try it 3: predict the size

Start with a 640×480 image (`img.size` $=$ `(640, 480)`).

1. `img.crop((100, 50, 300, 200)).size`

Try it 3: predict the size

Start with a 640×480 image (`img.size` $=$ `(640, 480)`).

1. `img.crop((100, 50, 300, 200)).size` $(200, 150)$
2. `img.thumbnail((256, 256))`, then `img.size`

Try it 3: predict the size

Start with a 640×480 image (`img.size` $=$ `(640, 480)`).

1. `img.crop((100, 50, 300, 200)).size` $(200, 150)$
2. `img.thumbnail((256, 256))`, then `img.size` $(256, 192)$
3. `img.resize((224, 224)).size`

Try it 3: predict the size

Start with a 640×480 image (`img.size` $=$ `(640, 480)`).

1. `img.crop((100, 50, 300, 200)).size` $(200, 150)$
2. `img.thumbnail((256, 256))`, then `img.size` $(256, 192)$
3. `img.resize((224, 224)).size` $(224, 224)$
4. `img.rotate(90, expand=True).size`

Try it 3: predict the size

Start with a 640×480 image (`img.size` $=$ `(640, 480)`).

1. `img.crop((100, 50, 300, 200)).size` $(200, 150)$
2. `img.thumbnail((256, 256))`, then `img.size` $(256, 192)$
3. `img.resize((224, 224)).size` $(224, 224)$
4. `img.rotate(90, expand=True).size` $(480, 640)$

Try it 4: what does `convert("L")` compute?

$$L = 0.299\,R + 0.587\,G + 0.114\,B$$

Pixel $(200, 100, 50)$: which gray value?

Try it 4: what does `convert("L")` compute?

$$L = 0.299\,R + 0.587\,G + 0.114\,B$$

Pixel $(200, 100, 50)$: which gray value?

$0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 59.8 + 58.7 + 5.7 = 124.2$

Pillow returns **124**.

Bonus: the sun pixel $(255, 210, 50)$ of the demo picture

Try it 4: what does `convert("L")` compute?

$$L = 0.299\,R + 0.587\,G + 0.114\,B$$

Pixel $(200, 100, 50)$: which gray value?

$0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 59.8 + 58.7 + 5.7 = 124.2$

Pillow returns **124**.

Bonus: the sun pixel $(255, 210, 50)$ of the demo picture

$76.245 + 123.27 + 5.7 = 205.215 \rightarrow$ **205**

Green has the largest weight because our eyes are most sensitive to green.

What can we learn about an image file?

Property	Pillow access	Why it matters
Dimensions	`img.size`, `.width`, `.height`	input-size planning, outliers
Colour mode	`img.mode`	same number of channels
File format	`img.format`	lossy or lossless
Info dictionary	`img.info`	DPI, ICC profile, comments
EXIF tags	`img.getexif()`	camera, time, orientation, GPS
File size	`os.path.getsize(path)`	storage and loading cost

Formats: JPEG vs. PNG vs. others

JPEG

- Lossy compression, small files
- No transparency
- Good for photographs
- Re-saving repeatedly degrades quality

PNG

- Lossless compression
- Supports an alpha channel
- Good for diagrams, masks, screenshots
- Larger files for photos

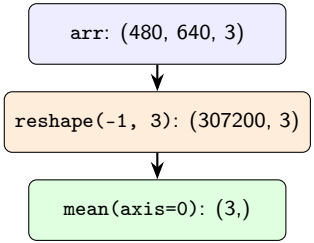
Data-science implication

Use lossless formats (PNG/TIFF) for *labels and masks*; lossy compression can alter label values.

Others you may meet: TIFF/GeoTIFF (scientific, satellite), WebP, BMP, GIF.

NumPy helpers that turn pixels into features

Function	Purpose
`arr.reshape(-1, 3)`	flatten (H, W, 3) into a list of pixels, one row each; -1: "work out this size"
`arr.mean(axis=0)`	average down the rows: one mean per column
`round(x, 2)`	keep 2 decimals



$480 \cdot 640 = 307{,}200$ pixels; three numbers summarise the colour of the picture.

Try it 5: `mean(axis=0)` by hand

Two pixels, three channels:

$$\texttt{a} = \begin{pmatrix} 10 & 20 & 30 \\ 30 & 40 & 50 \end{pmatrix}, \qquad \texttt{a.mean(axis=0)} = ?$$

Try it 5: `mean(axis=0)` by hand

Two pixels, three channels:

$$\texttt{a} = \begin{pmatrix} 10 & 20 & 30 \\ 30 & 40 & 50 \end{pmatrix}, \qquad \texttt{a.mean(axis=0)} = \, ?$$

Average each column: $\left(\frac{10+30}{2}, \frac{20+40}{2}, \frac{30+50}{2}\right) = (20,\ 30,\ 40)$.

For the demo picture, `reshape(-1, 3)` gives shape $(307200, 3)$, since $640 \cdot 480 = 307{,}200$.

Function cheat sheet

Family	Functions and attributes
Create	`Image.new`, `img.copy`
Read / write	`Image.open`, `img.save`
Inspect	`img.size`, `.width`, `.height`, `.mode`, `.format`, `getpixel`, `getexif`
Draw	`ImageDraw.Draw`, `rectangle`, `ellipse`, (`line`, `polygon`, `text`)
Transform	`convert`, `crop`, `resize`, `thumbnail`, `rotate`, `transpose`
To numbers	`np.array`, `Image.fromarray`, `reshape`, `mean`, `astype`
Tables	`pd.read_csv`, `df["col"]`, `pd.DataFrame`, `pd.concat`, `to_string`, `csv.writer`

Summary and lab preview

Remember

- Images are arrays: shape (H, W, C).
- `size` is (width, height).
- Pillow: make, ask, change, export.
- Image $\rightarrow$ numbers.

In the lab

- creating and saving images
- opening and inspecting them
- transforming them

Questions?